

Deuda Social en el Desarrollo Ágil de Software

Social Debt in Agile Software Development

Luis Romero
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
luisrom@unicauca.edu.co

Santiago Córdoba
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
santiago9739@unicauca.edu.co

César Pardo
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
cpardo@unicauca.edu.co
<https://orcid.org/0000-0002-6907-2905>

Eydy Suárez Brieve
Popular University of Cesar
GISICO Research Group
Valledupar, Cesar, Colombia
eydysuarez@unicesar.edu.co
<https://orcid.org/0000-0002-6296-0117>

Vladimir Villarreal
Technological University of
Panama
Grupo GITCE - UTP
Chiriquí, Panamá
vladimir.villarreal@utp.ac.pa
<https://orcid.org/0000-0003-4678-5977>

Abstract— In agile software development, suboptimal sociotechnical decisions can lead to adverse conditions that negatively impact the social and emotional well-being of teams. This phenomenon, known as social debt, manifests through deteriorated dynamics that undermine communication, collaboration, coordination, and team cohesion. This systematic literature review aims to analyze the current state of knowledge on social debt in agile contexts, identifying its causes, manifestations, consequences, and mitigation strategies. A total of 171 articles were reviewed, of which 21 were deemed relevant; five of these are primary studies. The findings reveal the presence of thirty community smells resulting from sociotechnical dysfunctions, along with nine critical factors influencing team effectiveness. Various types of non-technical debt were also classified, including process debt, people-related debt, organizational debt, and cultural debt. Additionally, three root cause elements and eight mitigation strategies were identified. The results underscore the need to develop specific metrics, adopt interdisciplinary approaches, and validate practical strategies for managing social debt. This review provides a solid conceptual and empirical foundation for understanding and addressing social debt in agile software development environments.

Keywords—Social debt, agile software development, systematic literature review, community odors, sociotechnical factors, organizational health

Resumen— En el desarrollo ágil de software, las decisiones sociotécnicas subóptimas pueden generar condiciones adversas que afectan el bienestar social y emocional de los equipos. Este fenómeno, denominado deuda social, se manifiesta en dinámicas deterioradas que comprometen la comunicación, la colaboración, la coordinación y la cohesión del equipo. Esta revisión sistemática de la literatura tiene como objetivo analizar el conocimiento actual sobre la deuda social en contextos ágiles, identificando sus causas, manifestaciones, consecuencias y estrategias de mitigación. Se identificaron 171 artículos, de los cuales 21 fueron considerados relevantes; cinco de ellos corresponden a estudios primarios. Los

resultados evidencian la existencia de treinta olores comunitarios derivados de disfunciones sociotécnicas, así como nueve factores críticos que afectan la efectividad del trabajo en equipo. Se clasificaron también distintos tipos de deuda no técnica (de procesos, de personas, organizacional y cultural), tres elementos generadores de causas y ocho estrategias de mitigación. Los hallazgos subrayan la necesidad de desarrollar métricas específicas, adoptar enfoques interdisciplinarios y validar estrategias prácticas. Esta revisión aporta una base conceptual y empírica para comprender y gestionar la deuda social en entornos ágiles.

Palabras clave—Deuda social, desarrollo ágil de software, revisión sistemática de literatura, olores comunitarios, factores sociotécnicos, salud organizacional

I. INTRODUCCIÓN

La deuda social (DS) en el desarrollo de software se refiere a las consecuencias acumuladas no gestionadas, derivadas de decisiones sociotécnicas subóptimas, que deterioran progresivamente la cultura organizacional, la comunicación interna, el bienestar del equipo y la calidad técnica del producto [1], [6]. A diferencia de la deuda técnica, la DS tiene su origen en dinámicas humanas mal gestionadas —como conflictos interpersonales, falta de transparencia, deficiencias en la coordinación y comunicación, o una distribución desigual de la carga de trabajo— que generan impactos tangibles sobre los miembros del equipo, manifestándose en estrés, desconfianza, irritabilidad y otros riesgos psicosociales que afectan tanto la salud física como emocional de los profesionales [7]. Uno de los síntomas más visibles de la DS es la aparición de olores comunitarios (*community smells*), entendidos como patrones disfuncionales en las relaciones o dinámicas de equipo. Si no se identifican y abordan a tiempo, estos pueden escalar y desencadenar crisis organizacionales [6], [13], por ejemplo: cuando los equipos priorizan de manera sistemática el

cumplimiento de plazos por encima del cuidado de las relaciones laborales, se acumula este tipo de deuda, lo que a largo plazo puede traducirse en estrés crónico, rotación de personal y fallos en el software [14], [7], [15].

En la última década, la adopción generalizada de metodologías ágiles ha intensificado este fenómeno. Dado que estas metodologías dependen de buena comunicación, cooperación y coordinación constantes, la falta de gestión adecuada de estos aspectos puede afectar negativamente los procesos de desarrollo. Diversos estudios revelan que muchos equipos ágiles operan con DS no diagnosticada, lo que se traduce en una disminución de la productividad y un incremento de los errores en el producto final [6], [9]. A pesar de su impacto tangible, la DS continúa siendo un fenómeno poco comprendido y escasamente abordado de forma sistemática. Asimismo, se identificó que persisten vacíos críticos en la literatura: no existe consenso sobre cómo medirla, las estrategias de mitigación carecen de validación empírica, y sus causas se abordan de manera fragmentada [13], lo que limita la capacidad de las organizaciones para intervenir eficazmente.

En este contexto, se justifica la realización de una revisión sistemática de la literatura (RSL) orientada a comprender las dinámicas no técnicas —especialmente aquellas relacionadas con fallas en la comunicación, coordinación y cooperación— que inciden en los equipos de desarrollo de software [16], [17]. Adicionalmente, esta RSL tiene como finalidad identificar las causas, efectos y consecuencias asociadas a la DS, consolidar las investigaciones existentes sobre el tema, reconocer los trabajos futuros sugeridos por la comunidad académica, y establecer un marco de prácticas necesarias para gestionar su acumulación en las organizaciones. Comprender la naturaleza y el impacto de la DS permite a las organizaciones diseñar estrategias más efectivas para promover entornos de trabajo sostenibles, saludables e innovadores, equilibrando las exigencias operativas con el bienestar de los equipos [13]. Este artículo está organizado de la siguiente manera: la Sección 2 describe el protocolo de investigación adoptado; la Sección 3 presenta el análisis detallado de los resultados; la Sección 4 discute los hallazgos en relación con la literatura existente; y la Sección 5 expone las conclusiones junto con las recomendaciones para investigaciones futuras.

II. PROTOCOLO DE INVESTIGACIÓN

Para este estudio, se recopilaron los principales artículos de investigación mediante una Revisión Sistemática de Literatura (RSL). Esta técnica permitió identificar, evaluar y analizar eficientemente artículos relacionados con problemas de investigación. Por lo tanto, una RSL facilita la recopilación de investigación primaria sobre un tema específico [5]. Para llevar a cabo el diseño de esta RSL, se siguió el protocolo propuesto por Kitchenham y Charters [18] y Budgen *et al.* [19], que proporciona directrices para la realización de una revisión sistemática. El protocolo comprendió las siguientes etapas: (i) definición de las preguntas de investigación; (ii) realización de la búsqueda de artículos relevantes; (iii) selección de artículos primarios aplicando criterios de inclusión y exclusión; (iv) evaluación de la calidad de los artículos primarios; y (v) extracción de información.

A. Definición de las preguntas de investigación

Para establecer los objetivos de búsqueda se utilizaron dos herramientas: la primera es propuesta por Doran con el enfoque conocido como SMART [20], el cual está conformado por un conjunto de criterios para definir objetivos y metas los cuales son: específicos, medibles, alcanzables, relevantes y temporales (en inglés Specific – S, Measurable – M, Achievable – A, Relevant – R, Time based - T), lo que aumenta la probabilidad para que los objetivos sean cumplidos. La segunda herramienta utilizada es el enfoque propuesto por Basili y Rombach [21] conocido como Goal-Question-Metric (GQM), el cual permitió establecer una estructura que apoya en el cumplimiento, avance y trazabilidad de los objetivos e intereses de la investigación propuestos, el enfoque sugiere tres niveles, el establecimiento de metas, la definición de preguntas, y finalmente, la definición de métricas para medir el avance así como los resultados de algún proyecto, esta última parte no se consideró en esta primera versión del RSL, se contemplará para futuros proyectos. La evaluación de la calidad de las preguntas de investigación se llevó a cabo siguiendo una adaptación del marco de trabajo definido por McCombes [22], el cual propone cuatro características específicas para evaluar la calidad de las preguntas de investigación, las características son: (i) centrado e investigable, (ii) factible y específico, (iii) complejo y discutible, y (iv) relevante y original, cada una de estas características está compuesta por criterios que garantizan su cumplimiento. A través del siguiente link: <https://zenodo.org/records/15654060>, se provee información complementaria acerca del proceso llevado a cabo para evaluar las preguntas de investigación formuladas.

A continuación, se pueden observar los objetivos (OB) incluidos en la RSL.

OB1: Identificar las investigaciones y propuestas existentes sobre la deuda social en el desarrollo de software ágil. Esto incluye la caracterización de los "dolores comunitarios", sus causas, consecuencias y los factores críticos que influyen en el desempeño de los equipos.

OB2: Explorar los principales enfoques y prácticas utilizados para la gestión y mitigación de la deuda social en equipos de desarrollo ágil. Esto abarca modelos, tácticas, marcos de trabajo y herramientas específicas propuestas en la literatura.

OB3: Determinar los desafíos actuales y las oportunidades de investigación futura en el ámbito de la deuda social en el desarrollo de software ágil. Esto implica identificar las brechas en el conocimiento, como la ausencia de métricas estandarizadas, la necesidad de enfoques interdisciplinarios y la exploración de tecnologías emergentes para su diagnóstico y manejo.

Con base en los objetivos propuestos, se plantearon las siguientes preguntas de investigación (PI), correspondientemente:

PI1: ¿Cuáles son las investigaciones o propuestas desarrolladas?

PI2: ¿Cuáles son los principales desafíos y oportunidades para la investigación futura?

P13: ¿Cuáles son las practicas o enfoques que abordan la gestión de la deuda social en el desarrollo de software ágil?

B. Ejecucion de la cadena de busqueda

En el protocolo de investigación también se utilizó el enfoque de revisión de literatura gris, también llamada literatura no convencional, y se refiere a la revisión de documentos que no se publican de manera tradicional. Estos documentos pueden ser informes de investigación, tesis, patentes, entre otros, y pueden ser difíciles de encontrar [4], esto permitió obtener una primera aproximación a los conceptos y términos abordados en la gestión de la deuda social, así como los problemas y retos a los que se enfrentan.

A partir de los términos identificados en la revisión de literatura gris, se definió la cadena de búsqueda básica: **“social debt” AND “agile software development”** la cual se aplicó a seis bases de datos científicas: Google Scholar, IEEE Xplore, SpringerLink, Scopus, Science Direct y Web Of Science. Es importante aclarar que la cadena de búsqueda básica debió ser adaptada de acuerdo con las especificaciones y configuraciones de cada base de datos. Asimismo, se tuvieron en cuenta los siguientes criterios o filtros de búsqueda de las diferentes bases de datos: (i) artículos en inglés, (ii) artículos exclusivos en áreas de conocimiento como: Computer Science, Software Engineering y Software Development, y (iii) artículos de revistas o actas de eventos científicos. Además, se consideró todo estudio dentro de la temática publicado entre los años 2013 y 2024, dado que el concepto de deuda social relacionado con el desarrollo de software y su gestión fue definido por primera vez en 2013 en el trabajo realizado por Tamburri *et al.* [1].

C. Selección de artículos primarios

En la Tabla 1 se puede observar el listado de los criterios de inclusión utilizados para identificar los estudios primarios que cumplieron con los lineamientos establecidos para la investigación. La Tabla 2 presenta el listado de los criterios de exclusión utilizados para identificar los estudios que no cumplieron con los lineamientos establecidos para la investigación.

TABLA 1. CRITERIOS DE INCLUSIÓN

Id	Criterio de inclusión (CI)
CI1	Aquellos artículos publicados entre 2013 y 2024.
CI2	Aquellos artículos que hayan sido escritos en inglés.
CI3	Aquellos artículos publicados en revistas o conferencias revisados por pares.
CI4	Aquellos artículos que en su tema principal proponga una solución para gestionar la deuda social.
CI5	Aquellos artículos que analizan las consecuencias de la deuda social y que sugieren o proponen una solución.

Acrónimos utilizados: Identificador (Id), Criterio de inclusión (CI).

TABLA 2. CRITERIOS DE EXCLUSIÓN

Id	Criterio de exclusión (CE)
CE1	Aquellos artículos que aborden el tema principal de investigación de manera superficial.
CE2	Aquellos artículos que aparecen duplicados en la búsqueda inicial de Google Scholar
CE3	Aquellos artículos que sean libros o capítulos de libros
CE4	Aquellos artículos con acceso restringido que requieran pagos externos o vinculaciones con terceros.
CE5	Tomaremos como base aquellos artículos encontrados inicialmente en Google Scholar, a partir de esta selección inicial, excluirémos cualquier artículo que aparezca repetido en otras bases de datos.

Acrónimos utilizados: Identificador (Id), Criterio de exclusión (CE).

Para la ejecución de dicho proceso y con el objetivo de reducir la subjetividad en el trabajo, se adaptó y ejecutó la

cadena de búsqueda básica a cada base de datos, asimismo, se verificó la ejecución de la búsqueda y los resultados obtenidos, resultado de este proceso se obtuvieron 171 artículos clasificados como encontrados. A partir de los artículos encontrados, 21 estudios fueron considerados como relevantes, y sólo 3 fueron considerados primarios. Considerando el número de artículos primarios resultantes, se optó por utilizar la estrategia definida por Jalali, S. y Wohlin, C. [2] denominada Snowballing Backward, también conocida como: “revisión bibliográfica retrospectiva”, aumentando el número de artículos, para un total de 5 estudios primarios que se pueden consultar en la Tabla 3. Cada uno de ellos fue elegido tras un proceso cuidadoso de revisión, considerando no solo su calidad metodológica, sino también el valor que aportan al entendimiento de la deuda social en equipos ágiles.

TABLA 3. ARTICULOS PRIMARIOS

Id	Nombre del artículo	Ref.
Artículos Snowballing		
EP1	“What is social debt in software engineering?”. Año 2013	[6]
EP2	“Beyond technical aspects: How do community smells influence the intensity of code smells?”. Año 2018.	[12]
Artículos primarios encontrados		
EP3	“The Second Vice is Lying, the First is Running into Debt.” Antecedents and Mitigating Practices of Social Debt: an Exploratory Study in Distributed Software Development Teams. Año 2021.	[17]
EP4	Community smells—The sources of social debt: A systematic literature review. Año 2023.	[11]
EP5	A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt. Año 2024.	[15]

Acrónimos utilizados: Identificador (Id), Referencia (Ref.).

III. ANÁLISIS DE LOS RESULTADOS

A continuación, se presentan los resultados obtenidos para cada una de las preguntas de investigación establecidas en esta revisión sistemática.

A. P11: ¿Cuáles son las investigaciones o propuestas desarrolladas?

A partir del análisis de los estudios primarios encontrados, se evidenció que el 60% (artículos: [6], [8], [9], [10], [11], [17], [23]) de los estudios se centran en: identificar, clasificar, analizar la deuda social y medir el impacto de las decisiones sociotécnicas en equipos de desarrollo de software subóptimos, así como: establecer factores críticos para la efectividad del trabajo en equipo, por ejemplo: Caballero *et al.* [6] menciona que la detección de olores comunitarios indica problemas dentro de los equipos de desarrollo de software, así como conflictos entre miembros del equipo o la falta de estándares claros. Con el tiempo, la persistencia de estos olores comunitarios resulta en la acumulación de deuda social, lo que afecta negativamente la colaboración, la eficiencia y la calidad del trabajo realizado en el proyecto de desarrollo de software. Asimismo, se mencionan nueve factores críticos del trabajo, así como: (i) contexto, (ii) cultura, (iii) comunicación, (iv) cooperación, (v) coordinación, (vi) conflicto, (vii) control, (viii) compromiso y (ix) cambio, cada uno de estos elementos puede tener un impacto en la manera en la que las interacciones sociales y las decisiones técnicas influyen en el desempeño del equipo de desarrollo de software. Al examinar las dinámicas de la comunidad bajo la perspectiva de estos factores, se puede obtener una comprensión más detallada de cómo estos olores afectan la dinámica interna del trabajo en equipo, y, por ende; en el rendimiento de los

equipos de desarrollo de software. Esta perspectiva puede proporcionar información valiosa sobre las causas y efectos de los olores de la comunidad en el entorno laboral y en la calidad del trabajo realizado por los equipos de desarrollo de software.

Por otra parte, en Caballero *et al.* [6] se identifican 30 olores comunitarios que surgen de situaciones sociales y organizativas adversas causadas por decisiones sociotécnicas incorrectas. Los efectos de estas decisiones incluyen interacciones sociales tensas, mala conducta personal y profesional, así como artefactos de software defectuosos como consecuencia. Los olores comunitarios identificados, junto con las causas y consecuencias se presentan en detalle en la Tabla 5.

Respecto a los artículos EP1 y EP5 (ver Tabla 3), éstos proponen diferentes soluciones, por su parte; Dressen *et al.* [13] proponen un modelo para reducir la deuda social basado en el modelo de las 3C's, es decir; comunicación, colaboración y coordinación, esto ayuda a categorizar los diferentes tipos de olores comunitarios dentro de una de las 3C's para así poder establecer el tipo de estrategia con el cual se procederá a gestionar y mitigar la deuda social. Este modelo también se centra en mejorar la comunicación y colaboración para reducir la deuda social por medio de una estrategia de entrevistas individuales con los miembros de más relevancia en el equipo de desarrollo de software ágil, si bien no se hace con todos los miembros del equipo, la recomendación es involucrar la mayor parte de individuos para un mejor entendimiento de la situación y una mejor contextualización del ambiente laboral. Por otro

lado, Saeeda *et al.* [11] investigan varios tipos de deuda no técnicas en el desarrollo de software, así como: (i) *la deuda de procesos* enfocados en los aspectos sociales, (ii) *deuda de la gente*, (iii) *deuda organizacional* y (iv) *deuda cultural*. Las deudas no técnicas se relacionan con problemas similares a la deuda técnica, pero surgen al no prestar atención a aspectos no técnicos del desarrollo. Descuidar estos factores puede llevar a resultados negativos en los equipos de desarrollo, incluyendo lo que se conoce como deuda social —algunas causas y estrategias para mitigarla son abordadas en la PI2—.

Asimismo, Saeeda *et al.* [11] identifican tres elementos generadores de causas en la deuda social: (i) limitaciones sociales, (ii) desafíos organizacionales y (iii) problemas comunitarios, a partir de los cuales se proponen ocho estrategias de mitigación: (1) análisis de redes sociales para una mejor comprensión de la deuda, hasta la (2) comunicación colaborativa, efectiva dentro del equipo, la (3) gestión de la composición del equipo y la mejora de decisiones arquitectónicas, (4) entornos de trabajo híbridos que combinan modalidades presenciales y remotas, (5) métricas específicas para la comunicabilidad de la arquitectura del software y la implementación de (6) marcos de trabajo para la mitigación de la deuda social, como CAFFEA, diseñado para empresas con productos complejos, asimismo, se destacan (7) tácticas arquitectónicas y frameworks sociotécnicos como DAHLIA, y la introducción de (8) herramientas específicas como GEEZMO para monitorear el estado de ánimo, CodeFace4Smell para

TABLA 5. OLORES COMUNITARIOS

No.	Nombre del olor	Causa	Consecuencia
1	Arquitectura por osmosis	Decisiones arquitectónicas sin una adecuada gestión de la información.	Arquitectura inestable y malentendidos.
2	Capucha de arquitectura	Dificultades en la cooperación entre equipos remotos.	Decisiones inconsistentes.
3	Nube negra	Falta de condiciones para una comunicación efectiva entre compañeros.	Disminución de la moral y productividad.
4	Cognición de clase	Desafío de entender clases y estructuras complejas.	Errores por interpretaciones distintas.
5	Código rojo	Clases complejas gestionadas por pocos desarrolladores.	Presión y estrés.
6	Distancia cognitiva	Diferencias entre desarrolladores.	Conflictos y malentendidos.
7	Desarrollo de recetario	Resistencia a nuevas metodologías y falta de innovación.	Falta de innovación.
8	Choque de DevOps	Conflictos entre los equipos de desarrollo y operaciones.	Afecta la integración y despliegue del software.
9	Desvinculación	Percepción de que el software está maduro y se entrega prematuramente.	Afecta la calidad.
10	Dispersión	Las correcciones fragmentan el equipo.	Dificulta la coordinación.
11	Disenso	Incapacidad de alcanzar consenso.	Problemas sin resolver, lo que afecta la calidad del software.
12	Hiper comunidad	Conexión excesiva dentro del equipo.	Retrasa las decisiones y disminuye la productividad.
13	Exceso de informalidad	Falta de protocolos formales.	Desbordamiento de información y afecta la eficiencia.
14	Isomorfismo institucional	Similitudes en los procesos.	Limita la innovación.
15	Arquitectura invisible	Decisiones arquitectónicas no se documentan adecuadamente.	Causa errores y falta de alineación.
16	Técnico sobrante	Comunicación deficiente entre desarrolladores y técnicos.	Disminuye la motivación y productividad.
17	Lobo solitario	Trabajo aislado de ciertos miembros.	Dificulta la cohesión y aumenta la duplicación de esfuerzos.
18	Arquitectura solitaria	Decisiones tomadas por no arquitectos.	Genera problemas de escalabilidad.
19	Recién llegados que no aportan	Dependencia excesiva de los nuevos miembros del equipo.	Afecta la productividad y cohesión.
20	Arquitectura obstruida	Falta de visión clara en sub-equipos.	Dificulta la implementación de cambios.
21	Silos organizacionales	Falta de coordinación entre tareas.	Lleva a duplicación de esfuerzos.
22	Escaramuza organizacional	Diferencias culturales entre equipos.	Afectan la gestión del proyecto.
23	Distancia de poder	Percepción de desigualdad.	Interrumpe la comunicación y afecta la toma de decisiones.
24	Miembros pedantes	Exigencia excesiva de conformidad.	Frustra a los compañeros.
25	Prima donnas	Resistencia al cambio y el trabajo aislado.	Afecta la innovación.
26	Silencio radiofónico o cuello de botella	Comunicación ineficiente en estructuras complejas.	Genera retrasos.
27	Villanía de compartir	No se comparte eficazmente el conocimiento.	Disminuye la cohesión y eficiencia.
28	Desafío de soluciones	Conflictos entre sub-equipos.	Impiden la resolución de problemas, afectando el progreso.
29	Distorsión temporal	Malentendidos después de cambios organizacionales.	Causando retrasos y problemas con los clientes.
30	Desaprendizaje	Empleados experimentados resisten el cambio.	Lleva a la pérdida de conocimiento y errores repetidos.

Acrónimos utilizados: Número (No.).

detectar causas en silos organizacionales, y YOSHI para mejorar la transparencia en comunidades de código abierto.

B. PI2: ¿Cuáles son los principales desafíos y oportunidades para la investigación futura?

Los principales desafíos en la investigación futura sobre la deuda social en el desarrollo de software ágil incluyen la comprensión de su naturaleza compleja de esta temática, el desarrollo de métricas efectivas para su evaluación, la integración de enfoques interdisciplinarios y la identificación de estrategias de mitigación. Estos desafíos requieren una profunda comprensión de los aspectos técnicos y sociales involucrados, así como la capacidad de medir y abordar la deuda social de manera oportuna y precisa. Dressen *et al.* [17] menciona que actualmente se desconoce qué causa exactamente el aumento de la deuda social y qué mecanismos ayudan a mitigar o disminuir sus efectos adversos. Caballero *et al.* [6] identifican 30 olores comunitarios, estos se presentan en la Tabla 5, junto con una posible causa y consecuencias. Los olores comunitarios en el desarrollo de software se deben entender como las distintas situaciones que pueden afectar la productividad y calidad del equipo.

Al analizar la Tabla 5, es posible observar la variedad de causas y consecuencias descritas en los olores comunitarios identificados, y tal parece que muchos de los “olores” y sus problemas derivan de deficiencias en: (i) Comunicación y Colaboración, (ii) Gestión de la Arquitectura y el Conocimiento, (iii) gestión del conocimiento, (iii) Estructurales y Organizacionales, (iv) Gestión del Talento y Dinámicas de Equipo Individuales, (v) Calidad y Entrega del Software. Con base en este análisis, es posible categorizar las causas y consecuencias en temas más amplios para comprender mejor las principales fuentes e impactos de estos “olores”. La Tabla 6 muestra los problemas y/o deficiencias en categorías genéricas con los números de entrada de los olores correspondientes a la Tabla 5.

TABLA 6. CAUSAS GENÉRICAS DE LOS OLORES

Id.	Categorías genéricas de las Causas	Id. del Olor (ver Tabla 5)
1	Problemas de Comunicación y Colaboración	2, 3, 6, 12, 13, 16, 17, 26, 27
2	Deficiencias en la Gestión de la Arquitectura y el Conocimiento	1, 4, 15, 18, 20, 30
3	Problemas Estructurales y Organizacionales	7, 8, 14, 21, 22, 23, 28, 29
4	Gestión del Talento y Dinámicas de Equipo Individuales	5, 10, 11, 19, 24, 25
5	Calidad y Entrega del Software	9, 11

Acrónimos utilizados: Identificador (Id).

A continuación, se presenta una descripción de cada una de las categorías y las relaciones inferidas con los olores relacionados de la Tabla 6:

1) *Problemas de Comunicación y Colaboración.* Esta categoría engloba los "olores" donde la falta de comunicación efectiva y la dificultad para trabajar en equipo son las causas centrales, llevando a malentendidos, ineficiencia y baja moral. La relación inferida surge por la comunicación deficiente, por ejemplo: la falta de canales adecuados o una transmisión clara de información y la falta de colaboración, por ejemplo: la incapacidad de los equipos o individuos para trabajar juntos de manera efectiva, son las raíces de muchos problemas. Estas fallas se manifiestan en inconsistencias en las decisiones, baja productividad, conflictos y retrasos.

2) *Deficiencias en la Gestión de la Arquitectura y el Conocimiento.* Esta categoría agrupa los “olores” donde la claridad, documentación y evolución del diseño del software y el conocimiento asociado son insuficientes. La relación inferida surge por una gestión inadecuada de la arquitectura, por ejemplo: decisiones no documentadas o tomadas por personas no cualificadas y una falta de claridad en el conocimiento técnico, por ejemplo: dificultad para entender estructuras complejas o resistencia a aprender llevan a arquitecturas inestables, errores constantes, problemas de escalabilidad y la imposibilidad de adaptarse a nuevas prácticas.

3) *Problemas Estructurales y Organizacionales.* Esta categoría aborda los desafíos derivados de la organización interna, los procesos y las dinámicas entre diferentes grupos o jerarquías. La relación inferida surge por la falta de coordinación entre departamentos o equipos, los conflictos inter-equipo (especialmente entre diferentes roles como desarrollo y operaciones), y una cultura organizacional rígida o jerárquica, por ejemplo: resistencia a nuevas metodologías o percepción de desigualdad, son causas directas de duplicación de esfuerzos, gestión de proyectos ineficaz y limitación de la innovación.

4) *Gestión del Talento y Dinámicas de Equipo Individuales.* Esta categoría se centra en cómo las actitudes, las responsabilidades individuales y la gestión de los miembros del equipo impactan la cohesión y el rendimiento. La relación inferida surge por la presión excesiva sobre pocos desarrolladores, la dependencia de nuevos miembros sin el apoyo adecuado, la incapacidad de llegar a consensos y las actitudes individuales problemáticas, por ejemplo: la resistencia al cambio o exigencia de conformidad, son factores que contribuyen directamente al estrés, la baja productividad, la frustración y una cohesión deficiente del equipo.

5) *Calidad y Entrega del Software.* Esta categoría se enfoca directamente en los “olores” que afectan el resultado final del desarrollo: la calidad del producto y su liberación. La relación inferida surge por la liberación prematura de software inmaduro y la persistencia de problemas técnicos o de diseño debido a la falta de resolución de conflictos son causas directas de la baja calidad del producto. Estos “olores” a menudo son una consecuencia de problemas en las categorías anteriores.

Al analizar las categorías de la Tabla 6, es posible observar que muchos “olores” no existen de forma aislada, por ejemplo: una comunicación deficiente (Categoría 1) puede llevar a una gestión de la arquitectura inadecuada (Categoría 2), lo que a su vez genera problemas estructurales (Categoría 3) y afecta la motivación del equipo (Categoría 4), culminando en una baja calidad del software (Categoría 5), por lo tanto, una deficiencia en la comunicación, a menudo exacerba otro tipo de problemas en la gestión de la arquitectura o las dinámicas del equipo. Por otra parte, abordar la dinámica interpersonal, podría ser un área clave para reducir la aparición y el impacto de olores en el desarrollo de software. En este sentido, abordar estos problemas y/o deficiencias, es crucial un enfoque integral u holístico que identifique no solo el olor superficial, sino también las causas profundas y las relaciones subyacentes entre ellos.

La Tabla 7 muestra las consecuencias y/o impactos en categorías genéricas con los números de entrada de los olores correspondientes a la Tabla 5. En resumen, al analizar cada olor

con su consecuencia directa, se refuerza la idea de que los "olores comunitarios" son síntomas de disfunciones sistémicas que impactan múltiples facetas de un equipo de desarrollo, desde su eficiencia operativa hasta la calidad de su producto y su capacidad de innovar.

TABLA 7. CONSECUENCIAS GENÉRICAS DE LOS OLORES

Id.	Categorías genéricas	Id. del Olor (ver Tabla 5)
1	Impacto en la Productividad y Eficiencia Operativa	3, 12, 13, 16, 17, 19, 21, 26, 27, 29
2	Impacto en la Calidad y Robustez del Software	1, 4, 15, 18, 20, 30
3	Impacto en la Cohesión y el Clima Laboral	7, 8, 14, 21, 22, 23, 28, 29
4	Impacto en la Innovación y Adaptabilidad	5, 10, 11, 19, 24, 25
5	Impacto en la Toma de Decisiones y Resolución de Problemas	9, 11

Acrónimos utilizados: Identificador (Id).

A continuación, se presenta una descripción de cada una de las categorías y las relaciones inferidas con los olores relacionados de la Tabla 5:

1) *Impacto en la Productividad y Eficiencia Operativa.* En esta categoría es posible observar que varios olores tienen consecuencias directas en la productividad, eficiencia, retrasos y duplicación de esfuerzos. Esto sugiere que la mayoría de los "olores" tienen un costo operativo tangible en el día a día del equipo. La falta de comunicación, la mala coordinación y las dinámicas de equipo ineficaces son los principales impulsores de esta pérdida de eficiencia.

2) *Impacto en la Calidad y Robustez del Software.* Olores como: Arquitectura por osmosis, Cognición de clase, Choque de DevOps, Desvinculación, Disenso, Arquitectura invisible, Arquitectura solitaria y Desaprendizaje, afectan directamente la calidad del software, la estabilidad de la arquitectura, la integración, el despliegue, la escalabilidad y la aparición de errores. Esto resalta la importancia de una buena gestión técnica y de procesos para asegurar un producto final sólido.

3) *Impacto en la Cohesión y el Clima Laboral.* Consecuencias como la disminución de la moral, conflictos, frustración, dificultad en la coordinación y la cohesión, indican que los "olores" tienen un fuerte componente humano y social. Un ambiente de trabajo negativo o desorganizado afecta directamente el bienestar de los miembros del equipo y su capacidad para colaborar eficazmente.

4) *Impacto en la Innovación y Adaptabilidad.* La falta de innovación es una consecuencia recurrente, a menudo ligada a la resistencia al cambio o a procesos rígidos. Esto sugiere que los "olores" no solo afectan el presente, sino que también comprometen la capacidad de la organización para evolucionar y mantenerse relevante.

5) *Impacto en la Toma de Decisiones y Resolución de Problemas.* Olores como: Disenso, Hiper comunidad, Distancia de poder y Desafío de soluciones muestran cómo las dinámicas internas pueden impedir la toma de decisiones efectiva y la resolución de problemas, lo que a su vez genera retrasos y afecta el progreso general del proyecto.

El análisis de la interacción entre las causas y las consecuencias de los "olores comunitarios" revela una compleja trama de retroalimentación negativa que afecta directamente la eficiencia y la calidad en los equipos de desarrollo de software. Asimismo, es posible observar que las deficiencias en la

comunicación y la colaboración actúan como catalizadores primarios, desencadenando una cascada de problemas que van desde la reducción de la productividad y la moral, hasta la inconsistencia en las decisiones y la fragmentación del equipo. Paralelamente, una gestión arquitectónica y del conocimiento inadecuada conduce a una inestabilidad técnica intrínseca en el producto, manifestada en errores recurrentes y dificultades de escalabilidad, lo cual, a su vez, puede generar mayor frustración y presión en el equipo. Finalmente, los conflictos internos, la resistencia organizacional al cambio y las dinámicas individuales desfavorables no solo limitan la innovación y el progreso del proyecto, sino que también culminan en una entrega de software de baja calidad, cerrando un ciclo vicioso donde los síntomas de la disfunción se perpetúan si las causas raíz no son abordadas de manera estratégica.

Según Caballero *et al.* [6] también se describe una serie de fases que permiten observar cómo surgen los "olores comunitarios" las cuales son: *Fase 1, Inicio:* los líderes toman decisiones sociotécnicas que generan un ambiente tenso, lo que puede llevar a malestar entre los miembros del equipo. A pesar de que este estrés comienza a manifestarse, no es fácilmente detectado por los gerentes. *Fase 2, Impacto directo de los problemas comunitarios:* el equipo experimenta más episodios de emociones negativas, lo que lleva a ignorar procesos importantes y reducir la calidad del trabajo. Aunque los efectos son más notorios, aún pueden pasar desapercibidos por los líderes. *Fase 3, Difusión del equipo:* los miembros afectados por estrés comienzan a influir negativamente en el resto del equipo, lo que provoca baja productividad y la acumulación de deuda social. Aunque los gerentes pueden detectar los problemas, la solución dependerá de sus habilidades y estrategias. *Fase 4, Impacto en la organización:* el deterioro de la calidad del software afecta a más equipos, y los líderes pueden identificar fallos técnicos que requieren atención urgente para evitar que los problemas se extiendan. *Fase 5, Problemas avanzados:* los usuarios externos comienzan a sufrir las consecuencias de los problemas, lo que afecta la reputación de la empresa y provoca pérdidas económicas debido a la pérdida de clientes.

En particular durante la Fase 3, denominada: "Difusión del Equipo", se destaca que los miembros del equipo que sufren episodios recurrentes de emociones negativas comienzan a manifestar sus estados psicológicos internos. Esto tiene un impacto directo en el resto del equipo, derivado de decisiones sociotécnicas que afectan a la cohesión grupal y contribuyen al aumento de la deuda social. Sin embargo, los líderes de los proyectos pueden detectar la presencia de estos efectos, pero el desarrollo de un método preciso para diagnosticar y solucionar los problemas depende de la experiencia del líder y los enfoques de gestión disponibles. Estos líderes tienen una oportunidad significativa para prevenir posibles daños organizativos al supervisar y mitigar periódicamente los efectos de los olores comunitarios dentro de los equipos de desarrollo de software.

En este sentido, se presentan oportunidades para investigar nuevas tecnologías y herramientas, explorar la relación entre la deuda social y el rendimiento del equipo, estudiar el papel de la cultura organizacional y evaluar el impacto a largo plazo de la deuda social en la dinámica del equipo y la calidad del software. Estas oportunidades pueden conducir a una comprensión más

completa de la deuda social en entornos ágiles y al desarrollo de estrategias más efectivas para su gestión y mitigación.

C. PI3: ¿Cuáles son las practicas o enfoques que abordan la gestión de la deuda social en el desarrollo de software ágil?

Dressen *et al.* [13] sugieren una estrategia para abordar la deuda social, fundamentada en el modelo de las 3C's. Este modelo enfatiza las deficiencias en comunicación, colaboración y coordinación mediante una tabla mencionada en el artículo. El enfoque de mitigación sugerido se concentra particularmente en mejorar la comunicación y la colaboración, destacando los siguientes aspectos:

a) *Comunicación honesta y abierta*, la cual es fundamental en equipos de desarrollo de software ágil distribuido para mitigar la deuda social. La comunicación honesta y abierta se define por su franqueza y transparencia, lo que ayuda a construir confianza mutua entre los miembros del equipo. Esta confianza facilita la resolución efectiva de conflictos sociales, ya que los miembros del equipo pueden abordar los problemas directamente. En última instancia, esta comunicación directa contribuye a crear un ambiente de trabajo constructivo y ayuda a reducir la acumulación de deuda social en el equipo.

b) *Intensidad de la comunicación relacionada con las tareas*, se refiere a la naturaleza altamente enfocada y orientada a objetivos de la interacción en el trabajo remoto. Las interacciones y la comunicación no son aleatorias, sino que están determinadas o preparadas de antemano para completar tareas específicas, reduciendo las distracciones. Este enfoque intensificado en la comunicación relacionada con las tareas puede ayudar a mitigar el efecto de los factores que contribuyen a la deuda social en equipos de desarrollo de software ágil distribuido (ASD).

c) *Intensidad de la colaboración relacionada con las tareas*, se refiere a cómo las interacciones durante el trabajo remoto están altamente enfocadas en tareas y objetivos específicos. Esto significa que todas las acciones colaborativas están orientadas a completar la tarea en cuestión, reduciendo las distracciones. Este enfoque intensificado en la colaboración relacionada con las tareas puede tener un efecto tanto en promover como en mitigar DS en equipos de desarrollo de software ágil distribuidos. Por un lado, puede conducir a una comunicación más directa y eficiente, pero por otro, puede resultar en una falta de interacción informal que es importante para la construcción de relaciones y la resolución de conflictos.

d) *Costos iniciales reducidos para iniciar interacciones o comunicaciones*, se refieren a la capacidad de establecer rápidamente un entorno para la colaboración y comunicación uno a uno sin restricciones. Esto es beneficioso en un contexto de trabajo remoto, ya que las salas de reuniones libres ya no son un recurso escaso y se puede configurar la comunicación sin casi ninguna limitación. Este factor ayuda a mitigar la deuda social en equipos de desarrollo de software ágil distribuido al facilitar la resolución de conflictos y promover una comunicación más directa y abierta entre los miembros del equipo.

IV. DISCUSIÓN

La deuda social en el desarrollo de software ágil se ha consolidado como un factor crítico que incide directamente en

la productividad operativa, la calidad del software resultante y la cohesión intrínseca de los equipos de desarrollo. Esta deuda se manifiesta de diversas formas, a menudo a través de lo que se ha denominado “olores comunitarios” Caballero *et al.* [6]. Estos “olores” actúan como señales tempranas, indicando disfunciones sociotécnicas subyacentes que pueden surgir de problemas persistentes en la comunicación, la coordinación, la gestión del conocimiento compartido, las dinámicas internas del equipo y la propia cultura organizacional. Por ejemplo, la presencia de conflictos no resueltos entre miembros del equipo o la ausencia de estándares claros para el trabajo colaborativo pueden ser precursores directos de la acumulación de deuda social.

A través del análisis de la literatura, es posible observar que, para abordar eficazmente esta deuda, se requiere la implementación de un enfoque holístico e integral, y a través de las estrategias de mitigación propuestas por diversos estudios, entre ellos: Dressen *et al.* [13] y Saeeda *et al.* [11], donde consistentemente enfatizan la necesidad de promover una comunicación honesta y abierta, fomentar una colaboración estrechamente orientada a tareas específicas y, crucialmente, reducir las barreras o fricciones que dificultan la interacción fluida y eficiente entre los miembros del equipo. Un modelo como el de las “3C” (comunicación, colaboración y coordinación) se presenta como una herramienta útil para clasificar los olores comunitarios y diseñar estrategias de intervención adecuadas. Además, se han identificado y propuesto diversas estrategias y herramientas específicas, desde el análisis de redes sociales para comprender las dinámicas internas hasta la implementación de marcos de trabajo sociotécnicos y el uso de herramientas que monitorean el estado de ánimo del equipo o detectan problemas organizacionales.

A pesar de los avances en la identificación y caracterización de la deuda social, existen desafíos significativos que limitan una gestión óptima, persistiendo una falta de claridad sobre las maneras de identificar, medir y cuantificar la de deuda social a través de las causas exactas que provocan su aumento, así como la determinación de cuáles son los mecanismos más efectivos para mitigar los efectos adversos que produce. La complejidad inherente a la interconexión de los factores sociales y técnicos, la urgente necesidad de desarrollar métricas efectivas y estandarizadas para su evaluación cuantitativa y cualitativa, y la importancia de integrar enfoques interdisciplinarios que combinen conocimientos de ingeniería de software, psicología organizacional y ciencias sociales, representan barreras significativas. No obstante, estos desafíos, a su vez, abren un amplio y prometedor campo de oportunidades para la investigación futura. La exploración de nuevas tecnologías para la detección temprana, el estudio del impacto a largo plazo en la resiliencia organizacional y el desarrollo de intervenciones basadas en evidencia podrían llevar a una comprensión más profunda y, en última instancia, a soluciones más robustas y sostenibles para mantener la salud social y la eficiencia técnica en los equipos de desarrollo de software ágil.

V. CONCLUSIONES Y TRABAJO FUTURO

Los hallazgos derivados de esta revisión sistemática de la literatura sugieren que la deuda social en el desarrollo de software ágil es un fenómeno crítico y multifacético. Asimismo,

se ha podido evidenciar que esta deuda no es un inconveniente de menor importancia, sino una acumulación de tensiones no técnicas, como el agotamiento emocional, la falta de reconocimiento profesional y el desequilibrio entre vida laboral y personal, que afectan directamente el bienestar del equipo. Además, que su manifestación se da a través de “olores comunitarios”, señales tempranas de disfunciones sociotécnicas subyacentes, que abarcan desde conflictos interpersonales y la falta de transparencia hasta una distribución desigual de las cargas de trabajo. La persistencia de estos “olores” tiene un costo operativo tangible, impactando la productividad, la eficiencia, la moral, la calidad del software y la capacidad de innovación.

A pesar de la creciente conciencia sobre el impacto de la deuda social, esta revisión subraya la persistencia de vacíos significativos en la literatura, específicamente, en que existe una falta de consenso sobre métricas estandarizadas que permitan cuantificarla de manera efectiva, y escasean las estrategias validadas empíricamente para su mitigación. Esta dispersión teórica y metodológica limita la capacidad de las organizaciones para implementar soluciones efectivas y oportunas. La complejidad inherente a la interconexión de factores sociales y técnicos, como la interacción entre una comunicación deficiente y una gestión arquitectónica inadecuada, requiere un enfoque integral que vaya más allá de la mera identificación de síntomas para abordar las causas profundas y las relaciones subyacentes entre los diferentes aspectos que la caracterizan.

En este contexto, la investigación futura se presenta con oportunidades valiosas para avanzar en el campo, entre ellas: (i) Es importante desarrollar herramientas de diagnóstico precisas que permitan a los líderes de proyecto no solo detectar la presencia de estos “olores” en fases tempranas, como la “difusión del equipo”, sino también medir su alcance y profundidad, superando la dependencia de la experiencia individual. Además, la (ii) validación empírica de estrategias de mitigación replicables es fundamental para establecer buenas prácticas que puedan ser adoptadas en diversos entornos de desarrollo ágil, esto incluye la evaluación de modelos como el de las “3C” (comunicación, colaboración y coordinación) y la eficacia de herramientas específicas que monitoreen el estado de ánimo o detecten problemas organizacionales. Finalmente, se identifica una valiosa oportunidad para explorar cómo las (iii) nuevas tecnologías, entre ellas la Inteligencia Artificial, pueden facilitar la detección temprana de los “olores comunitarios” y para investigar el (iv) impacto a largo plazo de la deuda social en la resiliencia organizacional. Un enfoque interdisciplinario, que combine conocimientos de ingeniería de software, psicología organizacional y ciencias sociales, será crucial para fomentar un ambiente de trabajo saludable y productivo, que no solo sea sostenible sino también propicio para la innovación y la creatividad.

AGRADECIMIENTOS

El profesor César Pardo agradece a la Universidad del Cauca el apoyo institucional brindado como profesor titular. El profesor Vladimir Villarreal es miembro del SNI en Panamá. La Profesora Eydy Suárez Brieva, profesora asociada de la Universidad Popular del Cesar, expresa su agradecimiento por el apoyo recibido a lo largo del desarrollo de este trabajo.

REFERENCIAS

- [1] D. A. Tamburri, P. Kruchten, P. Lago, y H. Van Vliet, “What is social debt in software engineering?”, en *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2013 - Proceedings*, 2013, pp. 93-96. doi: 10.1109/CHASE.2013.6614739.
- [2] S. Jalali y C. Wohlin, “Systematic literature studies: Database searches vs. backward snowballing,” en *Proceedings of the 2012 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, 2012, pp. 29–38. doi: 10.1145/2372251.2372257.
- [3] T. Dybå y T. Dingsøyr, “Strength of Evidence in Systematic Reviews in Software Engineering,” en *ESEM’08: Proceedings of the 2008 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, junio de 2008, pp. 178–187. doi: 10.1145/1414004.1414034.
- [4] L. Yang et al., “Quality Assessment in Systematic Literature Reviews: A Software Engineering Perspective,” febrero 01, 2021, *Elsevier B.V.* doi: 10.1016/j.infsof.2020.106397.
- [5] M. Ivarsson y T. Gorschek, “A method for evaluating rigor and industrial relevance of technology evaluations,” *Empirical Software Engineering*, vol. 16, no. 3, pp. 365–395, octubre de 2010, doi: 10.1007/s10664-010-9146-4.
- [6] E. Caballero-Espinosa, J. C. Carver, y K. Stowers, “Community smells—The sources of social debt: A systematic literature review,” *Information and Software Technology*, vol. 153, enero de 2023, doi: 10.1016/j.infsof.2022.107078.
- [7] E. S. Brieva, C. Pardo, y V. Villarreal, “Deuda Social y Riesgos psicosociales en entornos de desarrollo de software ágil: Análisis preliminar de una Revisión Sistemática de la Literatura,” en *2024 IEEE VII Congreso Internacional en Inteligencia Ambiental, Ingeniería de Software y Salud Electrónica y Móvil (AmITIC)*, 2024, pp. 1–8. doi: 10.1109/AmITIC62658.2024.10747594.
- [8] F. Palomba, D. A. Tamburri, A. Serebrenik, A. Zaidman, F. A. Fontana, y R. Oliveto, “How do community smells influence code smells?,” en *Proceedings - International Conference on Software Engineering*, 2018, pp. 240–241. doi: 10.1145/3183440.3194950.
- [9] D. A. Tamburri, P. Kruchten, P. Lago, y H. van Vliet, “Social debt in software engineering: insights from industry,” *Journal of Internet Services and Applications*, vol. 6, no. 1, diciembre. 2015, doi: 10.1186/s13174-015-0024-6.
- [10] A. Martini, V. Stray, y N. B. Moe, “Technical-, Social- and Process Debt in Large-Scale Agile: An Exploratory Case-Study,” en *Lecture Notes in Business Information Processing*, Springer Verlag, 2019, pp. 112–119. doi: 10.1007/978-3-030-30126-2_14.
- [11] H. Saeeda, M. Ovais Ahamd, y T. Gustavsson, “A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt,” *e-Informatica Software Engineering Journal*, vol. 18, no. 1, p. 240101, 2024, doi: 10.37190/e-inf240101.
- [12] E. A. C. Espinosa, “Understanding Social Debt in Software Engineering,” University of Alabama, Tuscaloosa, Alabama, USA, 2021. [Online]. Available: <http://ir.ua.edu/handle/123456789/8278>
- [13] T. Dreesen, P. Hennel, C. Rosenkranz, y T. Kude, “The second vice is lying, the first is running into debt. Antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams,” en *Hawaii International Conference on System Sciences*, 2021, pp. 6826–6835. doi: 10.24251/hicss.2021.818.
- [14] D. A. Tamburri, P. Kruchten, P. Lago, y H. van Vliet, “Social debt in software engineering: insights from industry,” *Journal of Internet Services and Applications*, vol. 6, no. 1, p. 10, 2015, doi: 10.1186/s13174-015-0024-6.
- [15] E. S. Brieva, J. E. Bejarano Guar, N. D. Urbano Palechor, y C. J. Pardo Calvache, “Olores comunitarios en comunidades de desarrollo de software: Revisión Sistemática de la Literatura,” *Ingeniería y Competitividad*, vol. 27, no. 2, junio de 2025, doi: 10.25100/iyv.v27i2.14826.
- [16] E. S. Brieva, C. de J. Pardo Calvache, y H. A. Ordoñez Eraso, “Instrumento para medir las percepciones de las comunidades de desarrollo de software respecto a la comunicación, coordinación y cooperación: Resultado de la validación mediante juicio de expertos,” *Inge CuC*, vol. 20, no. 1, pp. 116–134, mayo de 2024, doi: 10.17981/ingecuc.20.1.2024.10.
- [17] T. Dreesen, P. Hennel, C. Rosenkranz, y T. Kude, “The second vice is lying, the first is running into debt. Antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams,” *Proceedings of the Annual Hawaii International Conference on System Sciences*, vol. 2020-Janua, pp. 6826–6835, enero de 2021, doi: 10.24251/hicss.2021.818.
- [18] B. Kitchenham y S. Charters, “Guidelines for performing Systematic Literature Reviews in Software Engineering Version 2.3,” Durham, UK, julio de 2007. doi: 10.1145/1134285.1134500.
- [19] D. Budgen, M. Turner, P. Brereton, y B. Kitchenham, “Using Mapping Studies in Software Engineering,” vol. 2, 2007.
- [20] G. T. Doran, “There’s a S.M.A.R.T. Way to Write Management’s Goals and Objectives,” *Management Review*, vol. 70, pp. 35–36, 1981.
- [21] R. Van Solingen, V. Basili, G. Caldiera, y H. D. Rombach, “Goal Question Metric (GQM) Approach,” 2002. doi: 10.1002/0471028959.sof142.
- [22] S. McCombes, “Writing strong research questions | Criteria & examples,” agosto de 2023. [En línea]. Disponible en: <https://www.scribbr.com/research-process/research-questions/>
- [23] A. B. M. N. A. Talukder, “Understanding Social Debt in Software Engineering,” no. septiembre de 2022.